

Data Structures Lab: Menu-Driven Binary Search Tree (BST)

Experiment

Title: Implementation of Binary Search Tree (BST) Operations

Aim

To write a **menu-driven C program** to implement a Binary Search Tree and perform the following operations:

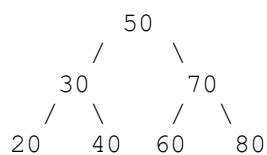
1. Insertion
2. Deletion
3. Searching
4. In-order traversal
5. Pre-order traversal
6. Post-order traversal
7. Exit

Theory

A **Binary Search Tree (BST)** is a binary tree in which:

- All nodes in the **left subtree** contain values smaller than the root.
- All nodes in the **right subtree** contain values greater than the root.
- The same rule is recursively applied to every node.

Example:



For this BST:

```
In-order    : 20 30 40 50 60 70 80
Pre-order   : 50 30 20 40 70 60 80
Post-order  : 20 40 30 60 80 70 50
```

Menu-Driven C Program

```
#include <stdio.h>
#include <stdlib.h>

// Structure of a BST node
struct Node {
```

```

    int data;
    struct Node *left;
    struct Node *right;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

// Insert a node into BST
struct Node* insert(struct Node *root, int data) {
    if (root == NULL) {
        return createNode(data);
    }

    if (data < root->data) {
        root->left = insert(root->left, data);
    }
    else if (data > root->data) {
        root->right = insert(root->right, data);
    }
    else {
        printf("Duplicate value %d is not allowed.\n", data);
    }

    return root;
}

// Search for a value
struct Node* search(struct Node *root, int key) {
    if (root == NULL || root->data == key) {
        return root;
    }

    if (key < root->data) {
        return search(root->left, key);
    }

    return search(root->right, key);
}

// Find the minimum value node
struct Node* findMin(struct Node *root) {
    while (root != NULL && root->left != NULL) {

```

```

        root = root->left;
    }

    return root;
}

// Delete a node from BST
struct Node* deleteNode(struct Node *root, int key) {

    if (root == NULL) {
        printf("Value %d not found.\n", key);
        return root;
    }

    // Search in left subtree
    if (key < root->data) {
        root->left = deleteNode(root->left, key);
    }

    // Search in right subtree
    else if (key > root->data) {
        root->right = deleteNode(root->right, key);
    }

    // Node found
    else {

        // Case 1: No child
        if (root->left == NULL && root->right == NULL) {
            free(root);
            return NULL;
        }

        // Case 2: Only right child
        else if (root->left == NULL) {
            struct Node *temp = root->right;
            free(root);
            return temp;
        }

        // Case 3: Only left child
        else if (root->right == NULL) {
            struct Node *temp = root->left;
            free(root);
            return temp;
        }

        // Case 4: Two children
        else {
            struct Node *temp = findMin(root->right);

            root->data = temp->data;

            root->right = deleteNode(root->right, temp->data);
        }
    }

    return root;
}

// In-order traversal

```

```

void inorder(struct Node *root) {

    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

// Pre-order traversal
void preorder(struct Node *root) {

    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

// Post-order traversal
void postorder(struct Node *root) {

    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

// Main function
int main() {

    struct Node *root = NULL;
    struct Node *result;

    int choice;
    int value;

    do {

        printf("\n=====\\n");
        printf("          BINARY SEARCH TREE\\n");
        printf("=====\\n");
        printf("1. Insert\\n");
        printf("2. Delete\\n");
        printf("3. Search\\n");
        printf("4. In-order Traversal\\n");
        printf("5. Pre-order Traversal\\n");
        printf("6. Post-order Traversal\\n");
        printf("7. Display All Traversals\\n");
        printf("8. Exit\\n");
        printf("=====\\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {

            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);

```

```

        root = insert(root, value);

        printf("Insertion completed.\n");
        break;

case 2:
    printf("Enter value to delete: ");
    scanf("%d", &value);

    root = deleteNode(root, value);
    break;

case 3:
    printf("Enter value to search: ");
    scanf("%d", &value);

    result = search(root, value);

    if (result != NULL) {
        printf("%d found in the BST.\n", value);
    }
    else {
        printf("%d not found in the BST.\n", value);
    }

    break;

case 4:
    if (root == NULL) {
        printf("BST is empty.\n");
    }
    else {
        printf("In-order Traversal: ");
        inorder(root);
        printf("\n");
    }
    break;

case 5:
    if (root == NULL) {
        printf("BST is empty.\n");
    }
    else {
        printf("Pre-order Traversal: ");
        preorder(root);
        printf("\n");
    }
    break;

case 6:
    if (root == NULL) {
        printf("BST is empty.\n");
    }
    else {
        printf("Post-order Traversal: ");
        postorder(root);
        printf("\n");
    }
    break;

```

```

        case 7:
            if (root == NULL) {
                printf("BST is empty.\n");
            }
            else {
                printf("\nIn-order Traversal    : ");
                inorder(root);

                printf("\nPre-order Traversal  : ");
                preorder(root);

                printf("\nPost-order Traversal : ");
                postorder(root);

                printf("\n");
            }
            break;

        case 8:
            printf("Program terminated.\n");
            break;

        default:
            printf("Invalid choice! Please try again.\n");
    }

    } while (choice != 8);

    return 0;
}

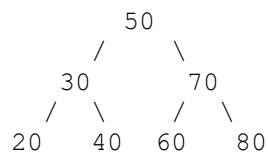
```

Sample Execution

Suppose the following values are inserted:

50 30 70 20 40 60 80

The BST becomes:



Sample Menu

```

=====
      BINARY SEARCH TREE
=====
1. Insert
2. Delete
3. Search
4. In-order Traversal
5. Pre-order Traversal
6. Post-order Traversal
7. Display All Traversals
8. Exit

```

=====
Enter your choice: 7

Output:

In-order Traversal : 20 30 40 50 60 70 80
Pre-order Traversal : 50 30 20 40 70 60 80
Post-order Traversal : 20 40 30 60 80 70 50

Search

Enter your choice: 3
Enter value to search: 60

60 found in the BST.

Searching for a value not present:

Enter your choice: 3
Enter value to search: 100

100 not found in the BST.

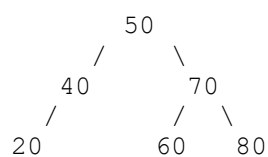
Deletion

For example:

Enter your choice: 2
Enter value to delete: 30

The node 30 has two children. It is replaced by its **in-order successor**, which is 40.

The resulting tree is:



In-order traversal:

20 40 50 60 70 80

Important BST Deletion Cases

Case	Node condition	Operation
1	Leaf node	Delete directly
2	Only left child	Replace node with left child
3	Only right child	Replace node with right child
4	Two children	Replace with in-order successor/predecessor

Complexity

Operation	Average Case	Worst Case
-----------	--------------	------------

Insertion	$O(\log n)$	$O(n)$
Search	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(n)$
Traversal	$O(n)$	$O(n)$

Note: The worst case occurs when the BST becomes skewed, for example when values are inserted in already sorted order.

Lab Exercise

Modify the above program to add:

1. Find the **minimum** element.
2. Find the **maximum** element.
3. Count the **total number of nodes**.
4. Count the **leaf nodes**.
5. Find the **height of the BST**.
6. Display the BST in a tree-like structure.